GROUP NAME: **LeVampire**

Group Members:

Gania Ruzzel Aaron R.

De Gracia Maria Isabel P.

Leander Lander Vince G.

Leonardo Ely May

Project Name: **Alacrity Auberge**

Project Description:

The project consists of a hotel reservation type website using different features that we learned from past lessons. The system includes a **login and registration page** for user authentication, a **dashboard** for both regular users and administrators, and supports **CRUD (Create, Read, Update, Delete)** operations for managing rooms, reservations, and user data. It also features a **file upload system** for uploading user profile pictures or room images. The website displays rooms of the hotel where you can book reservations and also the pricing rates of each room. The website's front end is designed using CSS and while PHP and MySQL handle the server-side logic and database interactions.

Project Features:

This web-based system is designed to manage hotel room reservations and customer interactions efficiently. It includes the following key modules and functionalities

1. **User Registration and Login**

Users can create an account by providing their personal details.

Passwords are hashed for security.

Login system with session-based authentication.

Role-based access for admin and regular users.

2. Dashboard

After login, users are redirected to a personalized dashboard.

Users see their reservations, profile details, and room availability.

Admins can view system statistics, user activity, and booking summaries.

3. Room Management (CRUD Operations)

Admins can Create, Read, Update, and Delete room listings.

Rooms include details such as room type, description, price, status, and images.

Rooms can be marked as “available” or “unavailable.”

4. Reservation System (CRUD Operations)

Users can Create new reservations by selecting check-in/out dates and room type.

Users can View and Cancel their reservations.

Admins can manage all reservations and update statuses (e.g., pending, confirmed, cancelled).

5. File Upload

Admins can upload room images via a file upload form.

The system checks file type and size to prevent malicious uploads.

6. Database Integration

All data is stored in a MySQL database (hotel_db) with tables for users, rooms, and reservations.

Foreign key relationships maintain data integrity between users and their bookings.

7. Session Management

PHP sessions are used to maintain user login states.

Only authenticated users can access the dashboard or perform bookings.

Sessions store user roles to enable access control.

Project Requirement:

1.) Login and Registration

- The Login and Registration feature provides a secure and user-friendly authentication system for the hotel reservation platform. During registration, users are required to input a username, email, and password, which undergo validation to ensure proper format, password match, and uniqueness of email. Passwords are securely hashed before being stored in the database. Upon successful registration, users are redirected to the login page. The login system verifies credentials by matching the entered email and hashed password against stored records, and initiates a session that stores the user's ID and role. This ensures only registered users can access protected pages and functionalities, while providing role-based control for further development such as admin features. The interface is styled with responsive layouts to enhance usability.

2.) File Upload

- The file upload feature allows users to securely upload files to the server through a web interface. It uses an HTML form with enctype="multipart/form-data" and handles the upload on the server side using PHP. The system checks for file validity such as file type, size, and extension before saving it to a designated directory to prevent malicious uploads. Uploaded files are stored in a structured location, and the file details (such as name and path) can optionally be saved in a database for tracking. This feature is useful for applications that require document submission, image uploads, or user-generated content.

3.) Database Connection

- The system connects to a MySQL database named hotel_db using PHP and the MySQLi extension, allowing secure interaction between the backend and the stored data. The database contains structured tables such as users, rooms, and reservations, which are essential for handling user authentication, room listings, and booking management. Relationships are enforced through foreign key constraints, for example, linking reservations to specific users and rooms ensuring data integrity. The connection is typically handled via a config.php file, which stores credentials like the hostname, username, password, and database name, and allows centralized, reusable access to the database across the application. This setup enables efficient CRUD operations and maintains a well-organized data structure for a hotel reservation system.

4.) Sessions

- The system uses PHP sessions to manage and maintain user authentication across different pages. Once a user successfully logs in, their unique ID and role are stored in session variables (e.g., $_SESSION['user_id'] and $_SESSION['role']), allowing the system to identify the user and provide access to appropriate resources without requiring them to re-enter credentials on each page. Sessions enable features such as user-specific content, access control, and persistent login states during the browsing session. Additionally, the session is used to restrict access to certain pages like the dashboard, ensuring that only logged-in users can access protected areas. This improves both security and user experience.

5.) CRUD Operations

- The system implements basic CRUD (Create, Read, Update, Delete) operations to manage essential data such as users, rooms, and reservations within the hotel management platform. Users can create accounts during registration, view and book available rooms, and administrators can add, edit, or delete room and reservation records directly from the system. Data is stored and retrieved from a MySQL database, with proper SQL queries handling each operation. These functionalities allow dynamic interaction with the database, making the system flexible and easy to maintain. CRUD operations are secured with input validation and database constraints to ensure data integrity and prevent unauthorized access or manipulation.